Project Write-Up

Group 1

System Explanation

What our system does

2048 is a sliding puzzle on a 4×4 grid. Every move pushes all the tiles in one direction, and two tiles with the same number merge into one worth double, adding that value to the score. The game then drops a new 2 or 4 on a random empty cell, and it ends when the board is full and nothing can merge. Space runs out fast, so you have to keep merging into larger tiles to stay alive, and reaching the 2048 tile counts as winning.

Our system plays the game by itself: it takes the current board, returns one legal move, and repeats until no move changes the board. We wrote the engine ourselves, including movement and merging, merge scoring, the random 2 (90%) or 4 (10%) spawn, and the game-over check.

What AI methods we use

We use two methods from the course. The first is reinforcement learning: TD(0) over an N-tuple network, which learns a value function $V(s)$ from self-play. The second is Expectimax, the chance-node version of the adversarial search we covered in class, which uses that value function to look ahead through the random spawns.

The pipeline

We train the value function separately and save it to a file. During a game the agent only reads that file, so the two halves are independent and nothing is learned while playing.

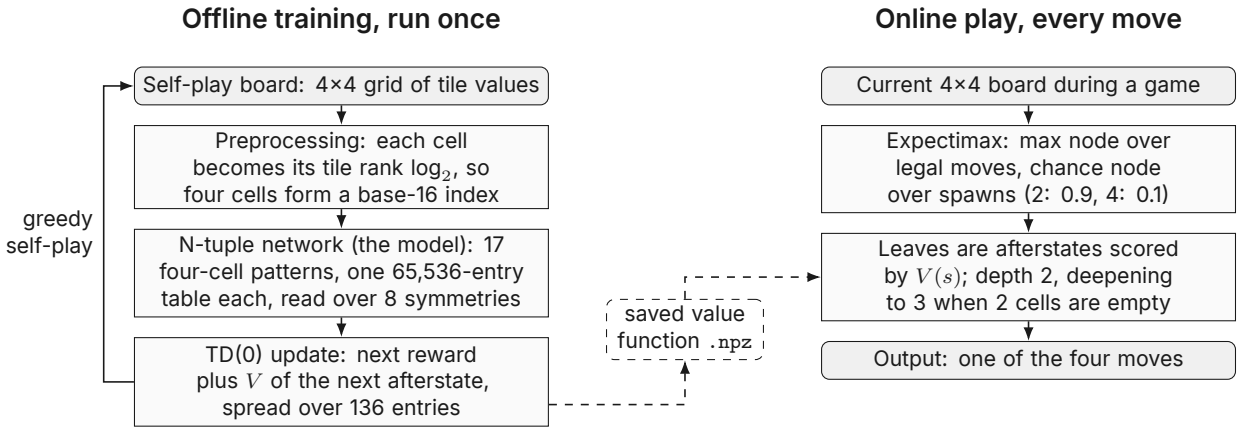

Figure 1. The two pipelines. Training writes the value function once, and play reads it on every move.

Both halves meet on afterstates, the boards you get after a move but before the spawn. A move in 2048 is deterministic and only the spawn is random, so the afterstate is the last position we can evaluate before luck enters. While training, we correct the value of the afterstate

we just left using the reward of the next move plus the value of the next afterstate, or zero if the game ended there. The search stops at that same point, so the learned V goes straight into Expectimax with nothing to convert. The eight board symmetries are what make training affordable, since every position we see updates all 136 shared entries at once.

Results

We recorded the average score of every block of 1,000 self-play episodes, then played the finished agent against two baselines for 30 games each in the same environment.

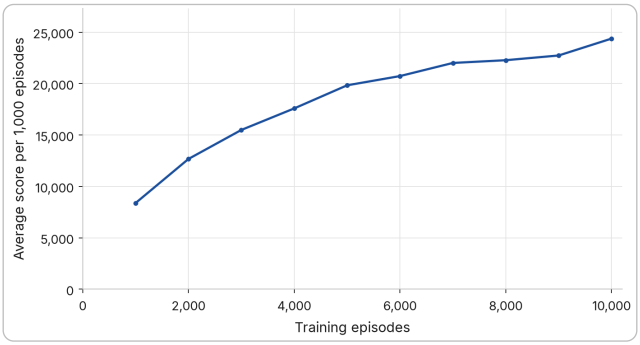

Figure 2. Average score per 1,000 training episodes. Self-play score rises from 8,384 to 24,382.

Agent	Games	Avg	Median	Best	Best tile	Reached 2048	ms/move
Random	30	1,178	990	3,076	256	0/30	0.0
Expectimax (heuristic)	30	14,724	15,698	27,308	2,048	5/30	8.8
Expectimax + RL	30	62,244	66,646	120,744	8,192	30/30	12.9

Table 1. The same search with a learned value function instead of a hand-written one, on the final code.

Swapping the hand-written evaluation for the learned one, inside the exact same search, multiplies the average score by 4.2 and takes the agent from 5 wins in 30 games to 30, at about 4 extra milliseconds per move.

Limitations and what we did about them

Three things still limit the agent. Learning starts slowly, because every TD update is divided among all 136 entries it touches, so the real step per weight is much smaller than α suggests; our first rate of 0.01 only reached about 15,500 after 10,000 episodes, and we settled on 0.05 after measuring both. Fixed depth 2 is also weakest exactly where games are lost, since on a nearly full board one bad spawn can leave no good move and a two-ply search sees only one spawn ahead. Two or fewer empty cells make the chance node small, so we deepen to 3 precisely there, which raises the cost from roughly 8 to 13 milliseconds per move and stays fast enough to watch. Finally, the value function knows nothing about board structure beyond its 17 local patterns, which is why we still need the search: playing greedily with no lookahead, the same network averages about 25,200 and wins half its games, against 62,244 and 30 out of 30 inside Expectimax. We kept the heuristic baseline for the same reason, to separate what learning contributes from what search does.

Feature Table

The table below summarizes the major components implemented for the 2048 environment, AI agents, reinforcement learning pipeline, and user interface.

Description	Platform	Completeness	Code	Author(s)	Notes
4×4 board and tile representation	Local	5	Python (NumPy)	João	Represents the board, tile values, and empty cells.
Tile movement and merging	Local	5	Python (NumPy)	Praneet	Implements movement in four directions and merges equal tiles when they collide.
Score, rewards, and game termination	Local	5	Python	Ryan	Handles rewards, score updates, legal-move checks, and game-over conditions.
Random tile spawning	Local	4	Python	Jayden	Spawns a 2 or 4 in a random empty cell after a legal move. Automated tests are missing.
Random agent	Local	5	Python	Jayden	Selects legal moves randomly and provides a baseline for evaluating stronger agents.
Heuristic Expectimax agent	Local	5	Python	João, Praneet, Ryan	Implements max and chance nodes, expected-value calculation, search depth, termination, and a hand-written evaluation function.
N-tuple network value function	Local	5	Python (NumPy)	João	Learns board values through tuple patterns, lookup tables, and board symmetries.
TD learning training pipeline	Local	4	Python (NumPy)	Praneet	Trains the N-tuple value function through self-play and saves periodic checkpoints. Automated tests are missing.
RL and Expectimax integration	Local	4	Python	Ryan	Uses the learned N-tuple value function inside Expectimax. Some automated tests are missing.
Agent evaluation and performance comparison	Local	5	Python	Jayden	Compares the random, heuristic Expectimax, and Expectimax + RL agents.
4×4 board and tile renderer	Local	5	Python (Pygame)	João	Displays the game board and tile values in the graphical interface.
Game loop and tile animations	Local	5	Python (Pygame)	Jayden	Controls gameplay and implements movement and merge animations.
Main menu and game header	Local	5	Python (Pygame)	Praneet, Ryan	Lets users select one of three agents or the comparison mode. Displays the title, score, best score, and restart control.
Automated testing	Local	3	Python	João, Praneet	Tests cover the N-tuple network and board renderer. Core game mechanics and agents still lack automated tests.

External Tools & Libraries

We implemented the project in Python and used the following libraries for numerical computation, visualization, testing, and the graphical interface:

- Matplotlib generates the TD-learning progress graph from the recorded average scores.
- Pytest runs the automated tests for the N-tuple network and user interface components.
- NumPy represents and manipulates the 4×4 board and stores the N-tuple lookup tables.
- Pygame provides the main menu, game board, score display, tile rendering, and animations.

We did not use an external dataset, API, or implementation of 2048. We wrote the game mechanics, Expectimax search, N-tuple network, TD-learning pipeline, and interface ourselves. Install the dependencies in `requirements.txt` with `pip install -r requirements.txt`; running the tests also requires Pytest.